

Request validation

Some of the simplest website (and account) hacks have been done through request alterations, even directly in the URL. Quite recently, someone exploited a YouTube vulnerability that involved request alteration. Before we go further, let us make it clear that request here is the method used by HTTP to communicate with the server/backend (the intricate details about the workings of HTTP are best left for another Fast Track).

What the person did was quite simple. When he tried to delete his own video, he simply altered the POST request generated at that point by replacing his video's ID with a target video ID. This caused the target video to be deleted and understandably created a lot of panic among youtubers. Something very similar could be easily executed on your website.

There are extensions and browser plugins that allow you to tamper with POST (and other) requests, like 'Tamper Data' for Firefox and 'Postman' for Chrome. Hence it is quite easy to exploit most websites using this method. To avoid this, simply associate a random value to the user's session when they log in (brownie points for you if you make this value hard to guess). Store a copy of this value on the server side too. With every POST request, include this value for validation, and reject any request where the two don't match. With that, you've just made your website a lot safer.

SQL Injection

For any poorly coded website, SQL injection is the easiest way to play havoc with their work. For most such websites, login information is handled in an SQL query. A normal user would enter their credentials which would be consequently authenticated. But a hacker could enter a very specific string that would change the logic of your authentication code to grant him access to the the first account of the database, which is usually the administrator.

Take the following code for example.

```
• SELECT id FROM users WHERE username = $username AND
password = $pwd;
```

If the hacker enters username as "1 OR 1=1; --" and any password, the statement is executed as

```
• SELECT id FROM users WHERE username = 1 OR 1=1;-- AND
password = any password;
```

The double dashes indicate the beginning of a comment, hence the password statement is ignored, and the user gets logged into someone else's account.